Binomial Tree Pricer

1 Overview

The binomial tree is a discrete-time approximation to the continuous-time Black-Scholes model. Instead of letting the stock price evolve continuously, we divide the option's life into N equal time steps and assume that at each step the stock can move in exactly one of two directions: up by a factor u, or down by a factor d. This creates a recombining tree of possible stock prices — "recombining" because an up-move followed by a down-move lands at the same price as a down-move followed by an up-move.

The tree has three advantages over the closed-form pricer:

1. **Generality**: it can price instruments where no analytical solution exists — most importantly, **American options** with early exercise, which will be supported in a future version of this library.
2. **Intuition**: the backward induction is a transparent, step-by-step calculation that makes the pricing logic easy to trace.
3. **Flexibility**: non-standard payoff structures can be incorporated by modifying the terminal or intermediate payoffs.

2 The CRR Parameterisation

The most widely used binomial tree is the **Cox-Ross-Rubinstein (CRR)** parameterisation, which chooses the up and down factors to match the volatility of the Black-Scholes model.

Let the time step be $\Delta t = T/N$. The CRR factors are:

$$u = e^{\sigma\sqrt{\Delta t}}, \qquad d = \frac{1}{u} = e^{-\sigma\sqrt{\Delta t}}$$

The choice $d = 1/u$ ensures the tree **recombines**: after any sequence of up and down moves, the order does not matter, only the count. This keeps the number of distinct nodes at time step i equal to $i + 1$ (not 2^i), making the algorithm computationally feasible.

The **risk-neutral probability** of an up-move is set so that the expected return under the risk-neutral measure matches the risk-free rate, net of dividends:

$$p = \frac{e^{(r-q)\Delta t} - d}{u - d}$$

For p to be a valid probability we need $0 < p < 1$, which holds as long as $d < e^{(r-q)\Delta t} < u$ — satisfied for any realistic combination of r, q, σ, and step count.

3 Building the Tree

At expiry (step N), the stock can take $N + 1$ distinct values, indexed by the number of up-moves $j \in \{0, 1, \ldots, N\}$:

$$S_T^{(j)} = S \cdot u^j \cdot d^{N-j}$$

where $j = 0$ gives the all-down outcome $S \cdot d^N$ (the lowest price) and $j = N$ gives the all-up outcome $S \cdot u^N$ (the highest price).

The terminal option values are simply the payoffs:

$$V_N^{(j)} = \max(S_T^{(j)} - K,\, 0) \quad \text{(call)}$$
$$V_N^{(j)} = \max(K - S_T^{(j)},\, 0) \quad \text{(put)}$$

4 Backward Induction

Once the terminal values are set, we work backwards through the tree. At each step, the value at a node is the discounted expected value of the next step's two children:

$$V_i^{(j)} = e^{-r\Delta t}\left[p\, V_{i+1}^{(j+1)} + (1-p)\, V_{i+1}^{(j)}\right]$$

Here, $V_{i+1}^{(j+1)}$ is the value after an up-move and $V_{i+1}^{(j)}$ is the value after a down-move. After N backward steps we reach the root, which contains the option price $V_0^{(0)}$.

In the implementation, this is vectorised over all j simultaneously:

```
V ← disc * (p * V[1:] + (1-p) * V[:-1])
```

Each iteration reduces the array length by 1. After N iterations, the single remaining value is the price.

5 Convergence to Black-Scholes

As $N \to \infty$, the binomial tree converges to the Black-Scholes closed-form price. The convergence rate is $O(1/N)$ for CRR — so doubling the number of steps halves the error. In practice, 200 steps gives accuracy within a few cents for typical options; 1000 steps reduces this to a fraction of a cent.

The error oscillates around the true value as N increases (a well-known feature of CRR), which means simply comparing prices at different step counts is more informative than trusting any single N.

```
N =  50:  error   0.04  (one step per week for a 1-year option)
N = 200:  error   0.01
N = 500:  error   0.004
N = 1000: error   0.002
```

Richardson extrapolation (averaging tree prices at N and $2N$ steps) can eliminate the leading-order error term and achieve $O(1/N^2)$ convergence, but this is not implemented here.

6 The `TreePricer` Class

`TreePricer` is a dataclass with a single configuration field: the number of time steps.

```python
from options_pricer import (
    BlackScholesModel, EuropeanOption, MarketData, OptionType, TreePricer,
)

md    = MarketData(spot=100.0, rate=0.05)
model = BlackScholesModel(vol=0.20)
call  = EuropeanOption(option_type=OptionType.CALL, strike=100.0, expiry=1.0)
put   = EuropeanOption(option_type=OptionType.PUT,  strike=100.0, expiry=1.0)

# Default: 200 steps
tree = TreePricer(n_steps=200)
```

```
print(tree.price(call, md, model))  # ~10.45
print(tree.price(put,  md, model))  # ~5.57
```

6.1 Comparing Accuracy Across Step Counts

```
from options_pricer import ClosedFormPricer

exact = ClosedFormPricer.price(call, md, model)

for n in [50, 100, 200, 500, 1000]:
    tr = TreePricer(n_steps=n).price(call, md, model)
    print(f"N={n:5d}  price={tr:.5f}  error={tr - exact:+.5f}")
```

6.2 With Dividends

The dividend yield q is incorporated through the risk-neutral probability $p = (e^{(r-q)\Delta t} - d)/(u - d)$. A positive dividend yield reduces p, reflecting the drag on the stock's expected return.

```
md_div = MarketData(spot=100.0, rate=0.05, div_yield=0.02)
print(TreePricer(n_steps=200).price(call, md_div, model))  # < 10.45 (dividends lower calls)
```

6.3 American Options

The binomial tree extends naturally to American options. During backward induction, each node represents a decision point: the holder can either continue holding (continuation value) or exercise immediately (intrinsic value). The American option value at node (i, j) is:

$$V_i^{(j)} = \max\Big(e^{-r\Delta t}\left[p\, V_{i+1}^{(j+1)} + (1-p)\, V_{i+1}^{(j)}\right],\; h_i^{(j)}\Big)$$

where $h_i^{(j)}$ is the intrinsic value at that node:

$$h_i^{(j)} = \max\Big(K - S_i^{(j)},\; 0\Big) \quad \text{(put)}, \qquad h_i^{(j)} = \max\Big(S_i^{(j)} - K,\; 0\Big) \quad \text{(call)}$$

In the implementation, the stock price at level i (measuring steps from the root) is reconstructed from the up/down factors:

$$S_i^{(j)} = S \cdot u^j \cdot d^{i-j}, \qquad j = 0, 1, \ldots, i$$

The backward induction loop becomes:

```
for k = 1, ..., N:
    v ← disc * (p * v[1:] + (1-p) * v[:-1])   # continuation
    if American:
        step ← N - k
        s_nodes ← S * u^j * d^(step-j)  for j = 0..step
        v ← max(v, intrinsic(s_nodes))          # early exercise
```

This single extra line per step is the only change from the European tree.

```
from options_pricer import AmericanOption, BlackScholesModel, MarketData, OptionType, TreePricer

md    = MarketData(spot=100.0, rate=0.05)
model = BlackScholesModel(vol=0.20)
am_put = AmericanOption(option_type=OptionType.PUT, strike=100.0, expiry=1.0)
```

```
# Early exercise premium: American > European by ~0.32
print(TreePricer(n_steps=500).price(am_put, md, model))  # ~5.89
```